Lab Project of Compilers - FIB UPC, 2026

The lab project will consist of building a compiler for an simplified imperative language named ASL (standing for *A Simple Language*).

The project will consist of:

- Writting a grammar for the language, and use ANTLR4 to generate a parser in C++
- Writting a semantic check module (in C++) that performs semantic analysis (i.e. variable type checking) on the tree produced by the parser on a given input program.
- Writting a code generation module (in C++) that generates low-level code for the input program. We will generate simple 3-address code that will run on a custom virtual machine called tVM (standing for *t-code Virtual Machine*).

In order to do the work, you will have some help:

- You will be using ANTLR-4, a powerful tool that will make it much easier to build your compiler.
- You will be provided with a basic working compiler (with just assignment instructions and sum operations) which you will need to complete until your ASL compiler is finished.
- You will also be provided with some auxiliary modules to help manage the data structures required to build the compiler.
- Finally, you will be provided with a list of ASL programs to incrementally test your compiler.

Warning

Auxiliary modules may change from one course to the next. If you are re-taking the course, your lab project may not work with this year modules. Exams will be compiled with current versions, so if you want to re-use code from last year, make sure you adapt it to be compatible with the latest versions of the auxiliary modules.

Here you will find further documentation about:

- The target language, ASL
- The provided compiler skeleton and auxiliary modules
- Summary of basic usage of ANTLR ParseTree API
- The description of tVM machine and the language it can execute
- The description of incremental program tests

You can also download the required code (also available at */assig/$USER/cl/files/CL-lab.tgz* in FIB labs):

- `Project base files`.

Project Evaluation

Although the project is to be carried out in teams of two people, the evaluation will be individual.

The project will be evaluated in two exams (one mid-term, the other final). Each exam will consist of implemeting an extension to the language supported by compiler developed along the course.

Exams from previous years are available for practicing.

ASL Description

You must build a compiler of ASL (*A Simple Language*), an imperative high-level programming language with the basic data types (integer, boolean, character, float), usual control structures (if-then-else, while, functions, ...), and arrays (one-dimension, basic type elements).

Here you will find a definition of the syntax of the language, and the expected behavior of the compiler in most frequent situations.

You will be provided example test suites with the errors that the compiler is expected to produce in each case.

Data types and variable declarations

Variables must be declared before being used. Variable declarations can appear exclusively at the begining of a function, and have the form:

```
var id1, id2, .., idn : type
```

Valid identifiers start by a letter, optionally followed by any string made of letters, digits, or underscores. Valid types for variables include basic -or primitive- types, as well as arrays.

Basic types are:

- `int` : Integer
- `float` : Real
- `bool` : Boolean
- `char` : Character

Arrays can contain elements of basic types only, and are defined as follows:

`array [<size>] of <type>`

(where `<size>` is an integer constant determining the number of elements and `<type>` is a basic type).

Examples of declarations:

```
var a, b : int
var a3, b_2, c3_x_ : bool
var x : float
var p, q : array [10] of char
```

Literal values of type `int` consist of one or more digits.

Literal values of type `float` consist of one or more digits followed by a decimal point, followed by one or more digits.

Literal values of type `bool` are `true` and `false`.

Literal values of type `char` are enclosed in single quotes, e.g. `'a'`, `'5'`, `'@'`. Escaped characters `'\n'`, `'\t'`, and `'\''` are allowed.

Functions, parameters, and results

Functions are declared using keywords `func` and `endfunc`. A comma-separated list of parameters (which maybe empty) is defined in parenthesis after the function name. For each parameter, name and type must be provided (i.e. brief parameter declaration of the form `x,y:int` is not allowed).

Finally, the return type of the function (if any) is declared after a colon. Functions can only return basic types.

All parameters are passed by value, except arrays, which are passed by reference.

Integer values can be passed as arguments to float parameters, in which case an implicit conversion to float is performed.

Declaration of local variables is located at the beggining of the function, following by the instructions in the function body.

Functions (either void or not) may include `return` instructions at any point, with an argument of the appropriate type (or nothing for void functions). Integer values can be returned inside float functions, and an implicit cast is performed.

Example of a function with boolean result:

```
func fun1 (a: array [10] of int, b: float) : bool
  var i : int
  var x : bool
  i = 5;
  if a[i]>b then
     return a[i+1]!=a[i-1];
  else
     x = false;
     return a[i-2]>0 or x;
  endif

endfunc
```

Example of a *void* function:

```
func fun2 (a: array [10] of int, i: int)
  var i : int
  i = 5;
  if a[i]>b then
    a[i+1] = a[i-1];
    return;
  else
     x = false;
  endif

  return;
endfunc
```

Functions that do not return a value can be called only as instructions. Functions returning a value can be called either inside expressions, or as instructions. In the later case, the return value is ignored.

Operators and precedence

ASL supports all usual aritmethic, relational, and boolean operators, with the following precedences and left associativity:

Operation	*Precedence*
`[]`	Highest
`not` `+` `-` (used as unary operators)	
`*` `/` `%`	
`+` `-`	
`==` `!=` `>` `>=` `<=` `<`	
`and`	
`or`	Lowest

As customary, these precedences may be overriden enclosing subexpressions in parenthesis.

All relational operators can be applied to all basic types, except booleans, that admit only `==` and `!=`.

Numeric types (float and int) can also be compared (after performing an implicit conversion to float).

Module operation (`%`) can only be applied to integer operands. The expected behaviour for negative operands is the same than in C++.

When integer values are aritmethically operated with float values, an implicit conversion to float is performed before the actual operation, and the result of the operation has type float.

Basic instructions

Basic instructions include assignment, `return` statement, and read/write statements. All basic instructions must end in a semicolon `;`

Assignment

```
var x,y,i,j : int
var a,b,c : array [10] of int
x = 0;
y = 2 + x;
a[i+1] = x*2 - b[j];
c = a;
```

Both primitive types and arrays can be assigned.

Integer values can be assigned to float variables after an implicit conversion (but not the other way round).

Input/output statements

- **Read** : reads a value from the standard input and stores it in given l-value. If the type of the input data does not match the l-value, the behaviour is undefined.

  Example:

  ```
  read x;
  ```

- **Write** : prints to the standard output the value of given expression.

  Example:

  ```
  write x+2;
  write '\n';
  ```

  Output statement `write` accepts also a string literal. This is the only use of string allowed in ASL.

  Example:

  ```
  write "The result is: ";
  write x+2;
  write '\n';
  ```

Return statement

Return statement causes current function to stop execution and return to the caller. If the function has a return value, the result expression must be given as argument to the `return` instruction.

This instruction can appear anywhere in a function. If the function contains paths not ending in a `return` instruction, the function will return an undefined value when reaching the end of its code.

Examples:

```
func f() : bool
   ...
   ...
   return x>0 or not b;
endfunc


func g()
   ...
   if (err!=0) then return; endif
   ...
endfunc
```

Control structures

ASL has two control structures: one for alternative composition and one for iterative composition:

- Alternative composition: alternative statements can be built with keywords `if`, `then`, `else`, and `endif`.

  The condition must be of boolean type. The `else` branch is optional.

  Example of an `if-then-else` construction

  ```
  if a>b then
     x = 0;
  else
     x = 2 + y;
  endif
  ```

  Examples of `if-then` constructions

  ```
  if x+1<0 then
     a=b;
  endif
  ```

- Iterative statements may be expressed with the `while-do-endwhile` construction

  The condition must be of boolean type.

  Example of `while-do-endwhile` construction

  ```
  while a>0 do
     s = s + a;
     a = a - 1;
  endwhile
  ```

Auxilary Compiler Modules

We will describe the modules you'll need to use following a skeleton code for your compiler

The main program of your compiler will look like the following code. You are supposed to read the comments and understand each step.

```
int main(int argc, const char* argv[]) {

  bool doTypeCheck=true, doCodeGen=true, doLLVM=false;
  std::string filename;
  for (int i=1; i<argc; ++i) {
    if (std::string(argv[i]) == "--noTypecheck") doTypeCheck=false;
    else if (std::string(argv[i]) == "--noCodegen") doCodeGen=false;
    else if (std::string(argv[i]) == "--genLLVM") doLLVM=true;
    else if (filename=="") {
      // it is not a valid option, must be the file name, make sure it is the first one
      filename = std::string(argv[i]);
    }
    else { // something unexpected came: Not a valid option, and a second filename
      std::cout << "Usage: ./asl [--noTypecheck|--noCodegen|--genLLVM] [<file.asl>]" <<
      return EXIT_FAILURE;
    }
  }

  // open input file (or std::cin) and create a character stream
  antlr4::ANTLRInputStream input;
  if (filename != "") {
    std::ifstream stream;
    stream.open(filename);
    if (stream.fail()) {
      std::cout << "Could not open file: " << filename << std::endl;
      return EXIT_FAILURE;
    }
    input = antlr4::ANTLRInputStream(stream);
  }
  else {            // read fron std::cin
    input = antlr4::ANTLRInputStream(std::cin);
  }

  // create a lexer that consumes the character stream and produces a token stream
  AslLexer lexer(&input);
  antlr4::CommonTokenStream tokens(&lexer);

  // create a parser that consumes the token stream, and parses it.
  AslParser parser(&tokens);

  // call the parser and get the parse tree
  antlr4::tree::ParseTree *tree = parser.program();

  // check for lexical or syntactical errors
  if (lexer.getNumberOfSyntaxErrors() > 0 or
      parser.getNumberOfSyntaxErrors() > 0) {
    std::cout << "Lexical and/or syntactical errors have been found." << std::endl;
    return EXIT_FAILURE;
  }

  // print the parse tree (for debugging purposes)
  // std::cout << tree->toStringTree(&parser) << std::endl;

  if (not doTypeCheck) {
    std::cout << "-- Early stop: no typecheck has been made." << std::endl;
    return EXIT_SUCCESS;
  }

  // auxililary classes we are going to need to store information while
  // traversing the tree. They are described below in this document
  TypesMgr       types;
  SymTable       symbols(types);
  TreeDecoration decorations;
  SemErrors      errors;

```

```
  // create a visitor that looks for variables and function declarations
  // in the tree and stores required information
  SymbolsVisitor symboldecl(types, symbols, decorations, errors);
  symboldecl.visit(tree);

  // create another visitor that will perform type checkings wherever
  // it is needed (on expressions, assignments, parameter passing, etc)
  TypeCheckVisitor typecheck(types, symbols, decorations, errors);
  typecheck.visit(tree);

  if (errors.getNumberOfSemanticErrors() > 0) {
    std::cout << "There are semantic errors: no code generated." << std::endl;
    return EXIT_FAILURE;
  }

  if (not doCodeGen) {
    std::cout << "-- Early stop: no code generated." << std::endl;
    return EXIT_SUCCESS;
  }

  // create a third visitor that will return the generated code
  // for each part of the tree, and will store it in 'mycode'
  CodeGenVisitor codegenerator(types, symbols, decorations);
  code mycode = std::any_cast<code>(codegenerator.visit(tree));

  // print generated code as output
  std::cout << mycode.dump() << std::endl;

  if (doLLVM) {
    std::string llvmStr = mycode.dumpLLVM(types, symbols);
    std::string llvmFileName;
    if (filename == "")
      llvmFileName = "output.ll";
    else {
      std::size_t slashPos = filename.rfind("/");
      std::size_t dotPos   = filename.rfind(".");
      llvmFileName = filename.substr(slashPos+1, dotPos-slashPos-1) + ".ll";
    }
    std::ofstream myLLVMFile(llvmFileName, std::ofstream::out);
    myLLVMFile << llvmStr << std::endl;
    myLLVMFile.close();
  }

  return EXIT_SUCCESS;
}
```

Code generated by ANTLR4

ANTLR4 is a parser generator that will create many of the code needed to build the compiler.

Given a grammar, (file *Asl.g4*) ANTLR4 will generate classes `AslLexer` and a `AslParser` that can be directly called from our main to obtain the parse tree of the target program.

Once we have the parse tree, we need to traverse it to perform type checking and code generation. ANTLR4 will also generate one abstract class called `AslVisitor` that has one `visit` method for each rule (or rule label) in the grammar. The *derived* class `AslBaseVisitor` implements these methods that walk the entire tree.

The `SymbolVisitor`, `TypeCheckVisitor`, and `CodeGenVisitor` are classes derived from `ASLBaseVisitor`. In each of them, the `visit` method for nodes that we want to process have been written, thus overwritting the empty methods in the base class. Each visitor will deal with some nodes and ignore some others, since each will do different things.

For instance:

- `SymbolVisitor` will only declare `visit` methods for nodes related to variable, parameter, or function declarations, and ignore all the rest.
- `TypeCheckVisitor` will declare `visit` methods for nodes related to expressions, assignments, and parameter passing, but ignore nodes about variable or function declarations.

- `CodeGenVisitor` will declare `visit` methods for nodes related to instructions and expressions, but ignore others.

Thus, to build a compiler with ANTLR4, we only need to write the grammar, a short main program like the previous example, and the needed `visit` methods of one (or more) derived `Visitor` classes that deal with the tree nodes we want to process.

Auxiliary Modules

The classes `TypesMgr`, `SymTable`, `TreeDecoration`, `SemErrors`, and the *Code Manager* module, containing class `code` and other related subclasses are used by our ASL compiler to store data about the program being compiled, and to propagate this information from one traversal to the following (e.g. Symbol declaration traversal will store information about which variables are declared and which type each of them has. Type checking traversal will use the type information to verify that operations are correctly performed, and code generation traversal will use information about the names and sizes of the variables to produce the right low-level code). Class `code` will be useful to contain the partially generated code, extend it with new instructions, and print it when it is complete. Class `counters` provides counters to keep track of used labels and temporals.

Type Manager

The Type Manager stores which data types have been seen in the target program (e.g. *bool*, *array of 10 char*, *function receiving one int and returning bool*) and offers some methods to manipulate them.

More information about Type Manager.

Symbol Table ¶

The Symbol Table stores which identifiers have been seen in the target program (variable names, parameter names, function names), in which scope have they appeared (i.e. in which function or code block), and associates them with a type (stored in Type Manager)

More information about Symbol Table.

Tree Decoration

The Tree Decoration module allows to store some information associated to specific nodes in the parse tree.

More information about Tree Decoration.

Code Manager

The Code Manager module contains several classes that ease the handling and combination of code fragments.

More information about Code Manager.

Semantic Errors

Semantic Errors module simplifies the handling of errors, associating errors to nodes in the tree.

More information about Semantic Errors.

Type Manager

This class stores information about all types existing in a program. A *type* stores information required to perform type checking.

Note that this includes not only primitive types, but programmer-defined types such as *array of 10 integers* or *function that receives a boolean and an array of 5 char and returns an integer*.

Thus, class `TypesMgr` class stores all seen types, and offers methods to find out information about them when we are type checking or generating code. For instance:

- *is type X an array?*
- *is type X the same than type Y?*
- *is type X compatible with type Y?*
- *which type have the elements of type X?* (when *X* is an array type, obviously)
- *which type returns type X?* (when *X* is a function type, obviously)

Here is the whole definition of `TypesMgr`. Check the comments to find out what each method does.

```
 1 ////////////////////////////////////////////////////////////////
 2 // Class TypesMgr: creates and stores types, and give access
 3 // to type information.
 4 // When a type is created the TypesMgr returns its TypeId.
 5 // The TypeId is the public way to work with types.
 6 // This TypeMgr works with five kinds of primitive types:
 7 // integer, float, boolean, character and void. Also it
 8 // recognizes two compound types: functions and fixed-size
 9 // arrays. Finally there exist a special type 'error'.
10
11 class TypesMgr {
12
13 public:
14
15   // The TypeId is an index in a vector
16   typedef std::size_t TypeId;
17
18   // Constructor
19   TypesMgr ();
20
21   // Methods to create a Type and return its TypeId
22   //   - Primitive and error types
23   TypeId createErrorTy     ();
24   TypeId createIntegerTy   ();
25   TypeId createFloatTy     ();
26   TypeId createBooleanTy   ();
27   TypeId createCharacterTy ();
28   TypeId createVoidTy      ();
29   //   - Compound types
30   TypeId createFunctionTy  (const std::vector<TypeId> & paramsTypes,
31                             TypeId                     returnType);
32   TypeId createArrayTy     (unsigned int               size,
33                             TypeId                     elemType);
34
35   // Accessors to work with primitive and error types
36   bool isErrorTy           (TypeId tid) const;
37   bool isIntegerTy         (TypeId tid) const;
38   bool isFloatTy           (TypeId tid) const;
39   bool isBooleanTy         (TypeId tid) const;
40   bool isCharacterTy       (TypeId tid) const;
41   bool isVoidTy            (TypeId tid) const;
42   bool isNumericTy         (TypeId tid) const;
43   bool isPrimitiveTy       (TypeId tid) const;
44   bool isPrimitiveNonVoidTy (TypeId tid) const;
45   bool isCompoundTy        (TypeId tid) const;
46
47   // Accessors to work with function types
48   bool                       isFunctionTy       (TypeId tid)     const;
49   const std::vector<TypeId> & getFuncParamsTypes (TypeId tid)     const;
50   TypeId                     getFuncReturnType  (TypeId tid)     const;
```

```
51   std::size_t                getNumOfParameters (TypeId tid)     const;
52   TypeId                     getParameterType   (TypeId tid,
53                                                  unsigned int i) const;
54   bool                       isVoidFunction     (TypeId tid)     const;
55
56   // Accessors to work with array types
57   bool         isArrayTy        (TypeId tid) const;
58   unsigned int getArraySize     (TypeId tid) const;
59   TypeId       getArrayElemType (TypeId tid) const;
60
61   // Methods to check different compatibilities of types
62   //   - structurally equal?
63   bool equalTypes      (TypeId tid1, TypeId tid2)     const;
64   //   - comparable with the relational operator op?
65   bool comparableTypes (TypeId tid1, TypeId tid2,
66                         const std::string & op)       const;
67   //   - tidFrom values can be copied to tidTo?
68   bool copyableTypes   (TypeId tidTo, TypeId tidFrom) const;
69
70   // Method to compute the size of a type (primitive type size = 1)
71   std::size_t getSizeOfType (TypeId tid) const;
72
73   // Methods to convert to string and print types.
74   std::string to_string (TypeId tidm) const;
75   void        dump      (TypeId         tid,
76                          std::ostream & os = std::cout) const;
77   // will return type name for basic types, element type name for arrays, 'none' for func
78   // useful for GenCode add_var and add_param
79   std::string to_string_basic (TypeId tidm) const;
80
81 };  // class TypesMgr
```

SymbolTable

The symbol table stores which *symbols* have been defined in the program (in ASL, that is just variable and function names) and associates each symbol with a *type* in `TypeMgr`.

The symbol table is also able to handle visibility scopes, so we can have separate tables for separate functions, or stacked tables for nested scopes.

Thus, class `SymTable` stores which symbols have been defined in each scope, and we can query it about which is the *type* of symbol, whether a symbol is defined in the current function, or (if it is defined twice because there is a nested scope) which is the definition we should use at each moment.

```
 1 //////////////////////////////////////////////////////////////
 2 // Class SymTable: stores the symbols declared in the program
 3 // along with the information associated with each one:
 4 //   - its class (the symbol can be a function, a parameter or
 5 //     a local variable)
 6 //   - its type (the TypeId returned by the TypesMgr)
 7 // The symbols are grouped in scopes. In the current version
 8 // of Asl there are two level of scopes: the global and the
 9 // local. The former for the symbols of function names, and
10 // the latter for symbols declared inside a function:
11 // parameters and local variables.
12 // The SymTable uses a 'stack' to keep the current available
13 // scopes that determines which symbols are visible and
14 // which are not. Entering in a function will push a new
15 // scope to the stack and exiting will pop the stack.
16
17 class SymTable {
18
19 public:
20
21   // The ScopeId is an index in a vector
22   typedef std::size_t ScopeId;
23
24   // Name of the Global Scope
25   static const std::string GLOBAL_SCOPE_NAME;
26
27   // Constructor
28   SymTable(TypesMgr & Types);
29   // Destructor
30   ~SymTable() = default;
31
32   // Manage the stack of scopes
33   //   - create a new empty scope and push its ScopeId in the stack
34   ScopeId pushNewScope  (const std::string & name);
35   //   - pop the stack of scopes
36   void    popScope      ();
37   //   - push a previously created scope sc and set it as current scope
38   void    pushThisScope (ScopeId sc);
39   //   - returns the current scope
40   ScopeId topScope      ()                          const;
41
42   // Methods to find an ident
43   //   - in the current scope (top of the stack)
44   bool    findInCurrentScope (const std::string & ident)            const;
45   //   - in the whole stack. Returns the number of scopes skipped to
46                           // find the symbol, or -1 if it is not found
47   int     findInStack        (const std::string & ident)            const;
48
49   // Adds a new symbol in the current scope
50   void addLocalVar  (const std::string & ident, TypesMgr::TypeId type);
51   void addParameter (const std::string & ident, TypesMgr::TypeId type);
52   void addFunction  (const std::string & ident, TypesMgr::TypeId type);
53
54   // Accessors to check the class of the symbol. If not found return false
55   bool isLocalVarClass  (const std::string & ident) const;
56   bool isParameterClass (const std::string & ident) const;
57   bool isFunctionClass  (const std::string & ident) const;
58
59   // Accessor to get the TypeId of a symbol. If not found return type 'error'
```

```
60   TypesMgr::TypeId getType (const std::string & ident) const;
61
62   // Check the existence of the "main" function
63   bool noMainProperlyDeclared() const;
64
65   // Given the name of a function, returns its TypeId
66   TypesMgr::TypeId getGlobalFunctionType (const std::string & ident) const;
67   // Given the names of a function and a local symbol, returns its TypeId
68   TypesMgr::TypeId getLocalSymbolType    (const std::string & funcName,
69                                           const std::string & ident) const;
70
71   // Print the symbols of a scope on the standard output
72   //   - the symbols of the current scope (top of the stack)
73   void printCurrentScope () const;
74   //   - the symbols of the whole stack
75   void print             () const;
76
77 };  // class SymTable
```

Tree Decoration

When traversing the tree for type checking, we may need to store information about a node in the tree that will be used later, either in type cheking, or in code generation.

Associating information to certaing nodes in the parse tree is called *decorating* the tree.

For example, an aritmethic expression (e.g. a sum) will have a type that will depend on the type of its operands (that may be other expressions, and in turn depend on the type of their operands...). Since expressions are not symbols, their type is not stored in the Symbol Table. Thus, when we climb the tree, we need to compute the type for each subexpression to allow nodes higher in the tree perform the checks they may need.

Class `TreeDecoration` stores information associated to tree nodes, and offers methods to retrieve it. Information that our compiler will need to keep about nodes is:

- The type of the node (e.g. for nodes corresponding to expressions).
- Whether the node is an l-value (e.g. it is at the left-hand side of an assignment).
- The scope to which the node belongs.

(note than not all nodes will be decorated with all this information. Some node will not be decorated at all)

Nodes are represented as `antlr4::ParserRuleContext *`, that is, pointers to nodes in the ParseTree built by ANTLR using our grammar rules.

```
1 ////////////////////////////////////////////////////////////////////
2 // Class TreeDecoration: the nodes of the parser tree generated
3 // by the antlr4 parser, whose base type is
4 // antlr4::ParserRuleContext *, can have different attributes.
5 // TreeDecoration groups all of them, and uses different
6 // ParseTreeProperty to save this information.
7 // Currently three kinds of attributes may be present:
8 //   - scope, for nodes like the program, or functions
9 //   - type, for expressions or type especification
10 //   - isLValue, for expressions
11 // Different visitors set and access these attributes:
12 //   - SymbolsVisitor     [TypeCheck phase 1]
13 //       * set and access the scope attribute
14 //       * set and access the type attribute (in type declarations)
15 //   - TypeCheckVisitor   [TypeCheck phase 2]
16 //       * access the scope attribute
17 //       * set and access the type attribute (in expressions)
18 //       * set and access the isLValue attribute (in expressions)
19 //   - CodeGenVisitor     [Code Generation]
20 //       * access the scope attribute
21 //       * access the type attribute
22
23 class TreeDecoration {
24
25 public:
26   TreeDecoration() = default;
27
28   // Getters:
29   SymTable::ScopeId getScope    (antlr4::ParserRuleContext *ctx);
30   TypesMgr::TypeId  getType     (antlr4::ParserRuleContext *ctx);
31   bool              getIsLValue (antlr4::ParserRuleContext *ctx);
32
33   // Setters:
34   void putScope    (antlr4::ParserRuleContext *ctx, SymTable::ScopeId s);
35   void putType     (antlr4::ParserRuleContext *ctx, TypesMgr::TypeId t);
36   void putIsLValue (antlr4::ParserRuleContext *ctx, bool b);
37
38 };  // class TreeDecoration
```

Code Manager

When generating code, our compiler will need to handle pieces of code created by different subtrees and combine them together to create more code (e.g. to create code for a *while*, the compiler will need to combine the code to compute the expression and the code of the instruction inside the loop with the right labels and jump instructions).

The Code Manager module provides a class `code` that can be used to handle this code combinations, and finally write it to the output file when it is complete. Objects of type `code` contain `subroutine` objects, which contain `instructionList` objects, which in turn contain `instruction` objects.

All these classes have methods to extend and combine them (e.g. add a `subroutine` to a `code`, or add an `instruction` to an `instructionList`, concatenate two `instructionList` objects, etc.)

Additionally, this module contains also a static class `counters` that manages counters to generate labels for jumps in *if* and *while* instructions, and well as for temporal registers.

```
/// predeclaration
class instructionList;
class LLVMCodeGen;

////////////////////////////////////////////////////////////////
/// Class instruction stores a VM instruction code with its operands

class instruction {
public:
  /// instruction codes
  typedef enum {_LABEL, _UJUMP, _FJUMP, _HALT, _PUSH, _POP, _CALL, _RETURN,
                _ADD, _SUB, _MUL, _DIV, _EQ, _LT, _LE, _NEG, _NOT, _AND, _OR, _FLOAT,
                _FADD, _FSUB, _FMUL, _FDIV, _FEQ, _FLT, _FLE, _FNEG,
                _LOAD, _ILOAD, _CHLOAD, _FLOAD, _XLOAD, _LOADX, _ALOAD, _LOADC, _CLOAD,
                _READI, _READF, _READC, _WRITEI, _WRITEF, _WRITEC, _WRITES, _WRITELN, _N

  /// instruction code
  Operation oper;
  /// arguments
  std::string arg1, arg2, arg3;

  /// constructor
  instruction(Operation op,
              const std::string &a1="", const std::string &a2="", const std::string &a3=

  /// destructor
  ~instruction();

  // concatenation of instruction+list (or instruction+instruction, via automatic coerti
  instructionList operator||(const instructionList &lst) const;

  /// ------ specific constructors for each instruction -------

  // create new instruction "a1 :"
  static instruction LABEL(const std::string &a1);
  // create new instruction "goto a1"
  static instruction UJUMP(const std::string &a1);
  // create new instruction "ifFalse a1 goto a2"
  static instruction FJUMP(const std::string &a1, const std::string &a2);
  // create new instruction "halt"
  static instruction HALT(const std::string &a1="");
  // create new instruction "pushparam a1"
  static instruction PUSH(const std::string &a1="");
  // create new instruction "popparam a1"
  static instruction POP(const std::string &a1="");
  // create new instruction "call a1"
  static instruction CALL(const std::string &a1);
  // create new instruction "return"
  static instruction RETURN();
  // create new instruction "a1 = a2 + a3"
  static instruction ADD(const std::string &a1, const std::string &a2, const std::string
  // create new instruction "a1 = a2 - a3"
  static instruction SUB(const std::string &a1, const std::string &a2, const std::string
```

```
54  // create new instruction "a1 = a2 * a3"
55  static instruction MUL(const std::string &a1, const std::string &a2, const std::string
56  // create new instruction "a1 = a2 / a3"
57  static instruction DIV(const std::string &a1, const std::string &a2, const std::string
58  // create new instruction "a1 = a2 == a3"
59  static instruction EQ(const std::string &a1, const std::string &a2, const std::string
60  // create new instruction "a1 = a2 < a3"
61  static instruction LT(const std::string &a1, const std::string &a2, const std::string
62  // create new instruction "a1 = a2 <= a3"
63  static instruction LE(const std::string &a1, const std::string &a2, const std::string
64  // create new instruction "a1 = a2 and a3"
65  static instruction AND(const std::string &a1, const std::string &a2, const std::string
66  // create new instruction "a1 = a2 or a3"
67  static instruction OR(const std::string &a1, const std::string &a2, const std::string
68  // create new instruction "a1 = a2 +. a3"
69  static instruction FADD(const std::string &a1, const std::string &a2, const std::strir
70  // create new instruction "a1 = a2 -. a3"
71  static instruction FSUB(const std::string &a1, const std::string &a2, const std::strir
72  // create new instruction "a1 = a2 *. a3"
73  static instruction FMUL(const std::string &a1, const std::string &a2, const std::strir
74  // create new instruction "a1 = a2 /. a3"
75  static instruction FDIV(const std::string &a1, const std::string &a2, const std::strir
76  // create new instruction "a1 = a2 ==. a3"
77  static instruction FEQ(const std::string &a1, const std::string &a2, const std::string
78  // create new instruction "a1 = a2 <. a3"
79  static instruction FLT(const std::string &a1, const std::string &a2, const std::string
80  // create new instruction "a1 = a2 <=. a3"
81  static instruction FLE(const std::string &a1, const std::string &a2, const std::string
82  // create new instruction "a1 = not a2"
83  static instruction NOT(const std::string &a1, const std::string &a2);
84  // create new instruction "a1 = - a2"
85  static instruction NEG(const std::string &a1, const std::string &a2);
86  // create new instruction "a1 = -. a2"
87  static instruction FNEG(const std::string &a1, const std::string &a2);
88  // create new instruction "a1 = float a2"
89  static instruction FLOAT(const std::string &a1, const std::string &a2);
90  // create new instruction "a1 = a2"
91  static instruction LOAD(const std::string &a1, const std::string &a2);
92  // create new instruction "a1 = a2" (where a2 is an integer constant)
93  static instruction ILOAD(const std::string &a1, const std::string &a2);
94  // create new instruction "a1 = a2" (where a2 is a character constant)
95  static instruction CHLOAD(const std::string &a1, const std::string &a2);
96  // create new instruction "a1 = a2" (where a2 is a float constant)
97  static instruction FLOAD(const std::string &a1, const std::string &a2);
98  // create new instruction "a1[a2] = a3"
99  static instruction XLOAD(const std::string &a1, const std::string &a2, const std::stri
100 // create new instruction "a1 = a2[a3]"
101 static instruction LOADX(const std::string &a1, const std::string &a2, const std::stri
102 // create new instruction "a1 = &a2"
103 static instruction ALOAD(const std::string &a1, const std::string &a2);
104 // create new instruction "a1 = *a2"
105 static instruction LOADC(const std::string &a1, const std::string &a2);
106 // create new instruction "*a1 = a2"
107 static instruction CLOAD(const std::string &a1, const std::string &a2);
108 // create new instruction "readi a1"
109 static instruction READI(const std::string &a1);
110 // create new instruction "readf a1"
111 static instruction READF(const std::string &a1);
112 // create new instruction "readc a1"
113 static instruction READC(const std::string &a1);
114 // create new instruction "writei a1"
115 static instruction WRITEI(const std::string &a1);
116 // create new instruction "writef a1"
117 static instruction WRITEF(const std::string &a1);
118 // create new instruction "writec a1"
119 static instruction WRITEC(const std::string &a1);
120 // create new instruction "writes 'string constant'"
121 static instruction WRITES(const std::string &a1);
122 // create new instruction "writeln"
123 static instruction WRITELN();
124 // create new instruction "noop" (not really needed)
125 static instruction NOOP();
126
127 // print instruction
128 std::string dump() const;
```

```
129 };
130
131
132 ////////////////////////////////////////////////////////////////
133 /// Class instructionList stores a list of instructions
134
135 class instructionList : public std::vector<instruction> {
136 public:
137   // constructor
138   instructionList();
139   // constructor from a single instruction
140   instructionList(const instruction &);
141   // destructor
142   ~instructionList();
143
144   // concatenation of lists (or list+instruction, via automatic coertion)
145   instructionList operator||(const instructionList &lst) const;
146
147   // print instructionList
148   std::string dump() const;
149 };
150
151
152 ////////////////////////////////////////////////////////////////
153 /// Class var stores a variable name and size
154
155 class var {
156 public:
157   std::string name;
158   std::string type;
159   size_t nelem;
160
161   var(const std::string &name, const std::string &type, size_t nelem=1);
162   ~var();
163
164   // print var
165   std::string dump() const;
166 };
167
168
169 ////////////////////////////////////////////////////////////////
170 /// Class subroutine stores information about a subroutine (local
171 /// vars, parametres, instructions, declared/used labels...)
172
173 class subroutine {
174 public:
175   /// list of local variables
176   std::list<var> vars;
177   /// list of params
178   std::list<var> params;
179
180   /// constructor and destructor
181   subroutine(const std::string &sname);
182   ~subroutine();
183
184   /// get subroutine name
185   std::string get_name() const;
186   /// add a local var to subroutine
187   void add_var(const var &v);
188   /// add a local var to subroutine
189   void add_var(const std::string &name, const std::string &type, size_t numelem=1);
190   /// add a parameter (size is always 1, since arrays are references)
191   void add_param(const std::string &name, const std::string &type, bool isarray=false);
192   /// add an instruction
193   void add_instruction(const instruction &inst);
194   /// add instruction list to current instructions
195   void add_instructions(const instructionList &lins);
196   /// set instruction list (overwritting current instructions)
197   void set_instructions(const instructionList &lins);
198
199   // print subroutine (params, vars, and instructions)
200   std::string dump() const;
201 };
202
203
```

```
204 ////////////////////////////////////////////////////////////////////
205 /// Class code stores a whole program (main plus subroutines)
206
207 class code {
208 public:
209   /// constructor and destructor
210   code();
211   ~code();
212
213   /// get most recently added subroutine (i.e. the one currently being processed)
214   subroutine& get_last_subroutine();
215   /// get subroutine by name
216   const subroutine& get_subroutine(const std::string &name) const;
217   /// add new subroutine
218   void add_subroutine(const subroutine &s);
219   /// get the list of subroutines (needed only in LLVMCodeGen)
220   const std::vector<subroutine> & get_subroutine_list() const;
221
222   // print code (all info for all subroutines)
223   std::string dump() const;
224   /// print the code in LLVM IR
225   std::string dumpLLVM(const TypesMgr & Types, const SymTable &Symbols) const;
226 };
227
228
229 ////////////////////////////////////////////////////////////////////
230 /// Class counters manages temporal and labels counters
231
232 class counters {
233 public:
234   // return id for new label or temp (id is a number, but returned as string
235   // to ease concatenation with other literals (e.g. "labelIF" + "4" -> "labelIF4")
236   static std::string newLabelIF();
237   static std::string newLabelWHILE();
238   static std::string newTEMP();
239
240   // reset individual counters
241   static void resetLabelIF();
242   static void resetLabelWHILE();
243   static void resetTEMP();
244
245   // reset label counters (IF and WHILE)
246   static void resetLabels();
247   // reset all counters (IF, WHILE, and TEMP)
248   static void reset();
249 };
```

Semantic Errors

The SemErrors module allows the compiler to record errors, along with related information (such as the line where they happened).

In this way, errors do not need to be emitted when they are found (which may lead to confusing order in the errors), but they can be printed ordered according to the line where they occur, once the analysis is completed.

Also, the class takes care of generating the error with the appropriate text, which would ease e.g. translation of error messages to languages other than English.

```
1  ////////////////////////////////////////////////////////////////
2  // Class SemErrors: this class contains methods that emit
3  // semantic error messages with their localization.
4  // It is used by the semantic visitors:
5  //   - SymbolsVisitor
6  //   - TypeCheckVisitor
7  // Semantic errors emitted are kept in a vector and when the
8  // typecheck finishes they will be printed (sorted by line/column number)
9
10 class SemErrors {
11
12 public:
13
14   // Constructor
15   SemErrors() = default;
16
17   // Write the semantic errors ordered by line number
18   void print ();
19
20   // Accessor to get the number of semantic errors
21   std::size_t getNumberOfSemanticErrors () const;
22
23   // Methods that store the error messages
24   //   node is the terminal node correspondig to the token IDENT in a declaration
25   void declaredIdent               (antlr4::tree::TerminalNode *node);
26   //   node is the terminal node correspondig to the token IDENT in an expression
27   void undeclaredIdent             (antlr4::tree::TerminalNode *node);
28   //   node is the terminal node correspondig to the token ASSIG
29   void incompatibleAssignment      (antlr4::tree::TerminalNode *node);
30   //   ctx is the node corresponding to the left expression
31   void nonReferenceableLeftExpr    (antlr4::ParserRuleContext *ctx);
32   //   node is the labeled operator token (referenced by op in the grammar)
33   void incompatibleOperator        (antlr4::Token *node);
34   //   ctx is the node corresponding to an array access
35   void nonArrayInArrayAccess       (antlr4::ParserRuleContext *ctx);
36   //   ctx is the node corresponding to the index expression in an array access
37   void nonIntegerIndexInArrayAccess (antlr4::ParserRuleContext *ctx);
38   //   ctx is the node corresponding to the expression
39   void booleanRequired             (antlr4::ParserRuleContext *ctx);
40   //   ctx is the node corresponding to the function identifier
41   void isNotCallable               (antlr4::ParserRuleContext *ctx);
42   //   ctx is the node corresponding to the function identifier
43   //   This error will not be emitted (productive functions can be called as procedures)
44   void isNotProcedure              (antlr4::ParserRuleContext *ctx);
45   //   ctx is the node corresponding to the identifier
46   void isNotFunction               (antlr4::ParserRuleContext *ctx);
47   //   ctx is the node corresponding to the function identifier
48   void numberOfParameters          (antlr4::ParserRuleContext *ctx);
49   //   pCtx is actual parameter node
50   //   n is the number of argument starting from 1
51   //   cCtc is the call node
52   void incompatibleParameter       (antlr4::ParserRuleContext *pCtx,
53                                     unsigned int n,
54                                     antlr4::ParserRuleContext *cCtx);
55   //   pCtx is actual parameter node
56   //   n is the number of argument starting from 1
57   //   cCtc is the call node
58   void referenceableParameter      (antlr4::ParserRuleContext *pCtx,
59                                     unsigned int n,
```

```
60                                  antlr4::ParserRuleContext *cCtx);
61   //   node is the terminal node correspondig to the token RETURN
62   void incompatibleReturn           (antlr4::tree::TerminalNode *node);
63   //   ctx is the read or write instruction
64   void readWriteRequireBasic        (antlr4::ParserRuleContext *ctx);
65   //   ctx is the instruction that needs a referenceable expression
66   void nonReferenceableExpression   (antlr4::ParserRuleContext *ctx);
67   //   ctx is the program node (grammar start symbol)
68   void noMainProperlyDeclared       (antlr4::ParserRuleContext *ctx);
69
70 };  // class SemErrors
```

Main methods to handle ANTLR ParseTree nodes

ANTLR generates a visitor method for each rule in the grammar. The visitor method looks like:

```
std::any visitRULE(AslParser::RULEContext *ctx)
```

where `RULE` is the name of the rule.

The parameter of each visitor method is a pointer `ctx` to an object of class `RULEContext`, which is derived from ANTLR class `ParseRuleContext`.

The pointer `ctx` allows access to information in the node currently being visited, and to its children.

For instance, if our grammar contains the rule:

```
expr : expr (MUL|DIV) expr
```

then our visitor will have a method:

```
std::any visitExpr(AslParser::ExprContext *ctx)
```

Inside this method we can, for instance:

- Check whether the node has a `MUL` child:

  ```
  if (ctx->MUL()) ...
  ```

- Get the string that matched the `MUL` regex:

  ```
  ctx->MUL()->getText()
  ```

  (not very useful for `MUL`, but very useful e.g. for `ID` or `INTVAL`)

- Check whether the rule has some `expr` child:

  ```
  if (ctx->expr())...
  ```

- Get how many `expr` children the rule has:

  ```
  ctx->expr().size()
  ```

  Note: Only when more than one `expr` are expected in the rule/subrule, `ctx->expr()` is a `std::vector<ParseRuleContext>`. Otherwise, `ctx->expr()` is *the* only `expr` child.

- Get the i-th `expr` child (counting from 0):

  ```
  ctx->expr(i)
  ```

- Get the total number of children of any kind:

  ```
  ctx->children.size()
  ```

- Get the i-th child of any kind:

  ```
  ctx->children[i]
  ```

- Both `ctx->expr()` and `ctx->children` can be iterated if needed:

  ```
  for (int i=0; i<ctx->expr().size(); ++i) visit(ctx->expr(i));
  for (auto ex : ctx->expr()) visit(ex);
  ```

```
for (int i=0; i<ctx->children.size(); ++i) visit(ctx->children[i]);
for (auto ch : ctx->children) visit(ch);
```

The last two iterations are equivalent to a simple call to `visitChildren(ctx)`.

t-Code Virtual Machine - Instruction set and operation manual

This page describes *t*VM, a virtual machine that runs *t*-code, which is a low level 3-address code.

Is is called *t*-code because the virtual machine has a very simple set of instructions and thus it is necessary to use a lot of *t*emporal registers to store intermediate results.

This document describes the instructions that the VM can process, as well as how to execute a program on it.

Executing a program in *t*VM

To execute a program, just run the VM with the name of the program file as an argument

```
./tvm myprogram.t
```

You can add the flag `--debug` to get details about the machine status while it executes your program.

```
./tvm myprogram.t --debug
```

*t*VM program structure

A program for *t*VM is just a list of functions. Exactly one of the functions must be named `main`

```
function main  ;;; this is the main program
    ...
endfunction

function f1    ;;; another function
    ...
endfunction

... etc
```

The string `;;;` starts a comment that will last until the end of the line. Comments can appear anywhere.

Function structure

Each function has three sections:

- Section `params` is optional, and declares the parameters the function expects to find in the stack.
- Each parameter is declared with its name and type. If the parameter is an array, its name and element type, followed by the keyword `array`. Array parameters are always passed by reference.
- Parameters are declared in the order they must be pushed.
- Parameters must be pushed by the caller.
- Function `main` can not have `params` section.
- All parameters use exactly one memory position.
- Section `vars` is optional, and lists the function local variables, with their types. If the variable is an array, the type of its elements plus the number of elements is listed.
- Finally, there is the function body which is a list of instructions. The execution of the function ends when a `return` instruction is found.

Example function:

```
function f1
   params
      p1 integer
      p2 float
      a1 integer array
   endparams
   vars
      v1 integer
```

```
      a2 character 10
   endvars

   ... instructions go here ...
   return
endfunction
```

Memory usage, variables and temporal registers

- *t*VM functions handle only parameters and local variables. All of them are stored in the stack.
- Function result value is handled as an additional parameter, that has to be pushed and popped by the caller.
- Function parameters use one memory position (32 bits).
- Variables can use any number of memory positions.
- Variables of sizes larger than 1 can be used as array base addresses and accessed with indexing instructions.
- Apart from stack memory, *t*VM has temporal registers, identified as `%1`, `%2`, `%3`, …
    - There is no limit to the number of used temporal registers.
    - Temporal registers are local to each function activation, and saved automatically by the VM between calls.

Instruction set

Flow control instructions

Instruction	*Effect*
`label label-name :`	Define a label in this position, to be jumped to.
`goto label-name`	Continue execution at given label.
`ifFalse addr goto label-name`	If the content of variable/temporal `addr` is zero continue execution at given label.

All labels are local to functions. No jumps between functions are allowed.

Assignment instructions

Instruction	*Effect*
`addr = 99`	Load integer value into variable/temporal `addr`.
`addr = 9.99`	Load float value into variable/temporal `addr`.
`addr = 'A'`	Load char value into variable/temporal `addr`. Escaped chars `'\t'` and `'\n'` are allowed.
`addr1 = addr2`	Copy value from variable/temporal `addr2` to variable/temporal `addr1`.

Aritmetic/relational/logical binary instructions

Instruction	*Effect*
`addr1 = addr2 operation addr3`	Perform `operation` with variables/temporals `addr2` and `addr3` and store result in variable/temporal `addr1`.

Valid operations are:	
`+ - * /`	Aritmetic operations with integer operands and integer result.
`+. -. *. /.`	Aritmetic operations with float operands and float result.
`== <= <`	Comparison operators with integer operands and integer (0/1) result.
`==. <=. <.`	Comparison operators with float operands and integer (0/1) result.
`and or`	Logical operators with integer (0/1) operands and integer (0/1) result.

Aritmetic/logical unary instructions

Instruction	*Effect*
`addr1 = operation addr2`	Perform unary `operation` on variable/temporal `addr2` and store result in variable/temporal `addr1`.
Valid operations are:	
`-`	Sign change with integer operands and integer result.
`-.`	Sign change with float operands and float result.
`not`	Logical negation with integer (0/1) operands and integer (0/1) result.
`float`	Integer to float conversion with integer operand and float result.

Assignment instructions with indirections or indexing

Instruction	*Effect*
`addr1[addr2] = addr3`	Copy content of variable/temporal `addr3` to `addr2` positions after `addr1` memory address. If `addr1` is a variable or parameter, its address is used. If it is a temporal, it is assumed to contain a pointer to the base address to be used.
`addr1 = addr2[addr3]`	Copy to variable/temporal `addr1` the content of `addr3` positions after `addr2` memory address. If `addr2` is a variable or parameter, its address is used. If it is a temporal, it is assumed to contain a pointer to the base address to be used.
`temp = &var`	Copy to temporal `temp` the memory address of `var`.
`addr1 = *temp`	Copy to variable/temporal `addr1` the content of memory address contained in temporal `temp`.
`*temp = addr1`	Copy the content of variable/temporal `addr1` to the memory address contained in temporal `temp`.

Stack instructions

Instruction	*Effect*
`pushparam addr1`	Push content of variable/temporal `addr1` into the stack.
`pushparam`	Push a zero into the stack (typically to reserve space, e.g. for a function result).
`popparam addr1`	Get content of stack top into `addr1` and pop it from the stack.
`popparam`	Pop top of the stack, ignoring its value.

Function calling instructions

Instruction	*Effect*
`call fname`	Execute function `fname`.
`return`	End function execution and return control to caller.

Input/output instructions

Instruction	*Effect*
`readi addr1`	Read an integer value and store it in `addr1`.
`readf addr1`	Read an float value and store it in `addr1`.
`readc addr1`	Read an character value and store it in `addr1`.
`writei addr1`	Write content of variable/temporal `addr1` as integer.
`writef addr1`	Write content of variable/temporal `addr1` as float.
`writec addr1`	Write content of variable/temporal `addr1` as character.
`writes "somestring"`	Write a string literal.

Instruction	*Effect*
`writeln`	Write a newline.

Example programs

Factorial

```
;;; This program reads an integer and computes
;;; its factorial using a recursive function

function main
  vars
    x integer        ;;; int x, y
    y integer
  endvars

  readi x            ;;; read x
  pushparam          ;;; make space for function result
  pushparam x        ;;; pass parameter x
  call fact          ;;; y = fact(x)
  popparam           ;;; remove passed parameter x
  popparam y         ;;; pop result and store in y
  writei y           ;;; print y
  writeln
  return
endfunction

function fact
  params
    _result integer  ;;; _result is an internal variable used to
                     ;;; access the result position in the stack
    n integer        ;;; received parameter
  endparams

  vars
    f integer        ;;; local variable
  endvars

  %1 = n == 0             ;;; compute %1 = (n==0)
  ifFalse %1 goto else1
  f = 1                    ;;; n==0, so f=1
  goto endif1

  label else1 :            ;;; n!=0, prepare recursive call
  pushparam                ;;; space for function result
  %2 = n - 1               ;;; %2 = n-1
  pushparam %2             ;;; pass n-1 as parameter
  call fact                ;;; f = fact(n-1)
  popparam                 ;;; remove passed parameter n-1
  popparam f               ;;; pop result and store it in f
  f = n * f                ;;; f = n * f
  label endif1 :
  _result = f              ;;; store f in space reserved for result
  return
endfunction
```

Example – Computing number *e*

```
;;; This program computes number "e" with a precision "eps"
;;; using the formula e = sum 1/(i!)  for all i>0

function main
  vars
    e float          ;;; float e, eant, eps, f
    eant float
    eps float
    f float
    i integer        ;;; int i
  endvars
```

```
  eant = 0.0
  e = 1.0
  eps = 0.00001
  f = 1.0
  i = 1

  label while1 :     ;;; while (eps < e-eant)
  %1 = e -. eant
  %1 = eps <. %1
  ifFalse %1 goto endwhile1
  eant = e

  %2 = 1.0 /. f       ;;; e = e + 1.0/f (e and f are float, note float
  e = e +. %2        ;;; operations "+." and "/.")

  i = i + 1         ;;; i = i + 1 (i is int -> integer addition "+")

  %3 = float i       ;;; f = f * i
  f = f *. %3        ;;; (i is converted to float before
                     ;;; performing a float product "*.")
  goto while1
  label endwhile1 :   ;;; endwhile

  writef e
  writeln

  return
endfunction
```

Example - Reversing a vector

```
;;;  This program reads a number x, fills an array of 10
;;;  elements with values x to x+9, reverses the array,
;;;  and finally prints the array elements

function main
  vars
    x integer             ;;; int x, i, n
    i integer
    n integer
    a integer 10          ;;; int a[10]
  endvars

  readi x
  n = 10                  ;;; n=10

  ;;; fill array with numbers from x to x+9
  i = 0                   ;;; i=0
  label for1 :            ;;; while (i<n)
  %1 = i < n
  ifFalse %1 goto endfor1

  %2 = x + i              ;;; a[i] = x+i
  a[i] = %2

  %2 = 1                  ;;; i = i + 1
  i = i + %2
  goto for1               ;;; endwhile
  label endfor1 :

  ;;; reverse array, swapping x[i] with x[n-i-1] for all i<n/2
  i = 0                   ;;; i=0
  label for2 :            ;;; while (i<n/2)
  %1 = 2
  %2 = n / %1
  %3 = i < %2
  ifFalse %3 goto endfor2
  x = a[i]                ;;; x = a[i]
  %1 = 1                  ;;; a[i] = a[n-1-i]
  %1 = n - %1
  %1 = %1 - i
  %2 = a[%1]
```

```
  a[i] = %2

  a[%1] = x               ;;; a[n-1-i] = x

  %2 = 1                  ;;; i = i + 1
  i = i + %2
  goto for2               ;;; endwhile
  label endfor2 :

  ;;; print array elements
  i = 0                   ;;; i=0
  label for3 :            ;;; while (i<n)
  %1 = i < n
  ifFalse %1 goto endfor3

  %2 = a[i]               ;;; write a[i]
  writei %2
  %2 = ' '                ;;; write ' '
  writec %2

  %2 = 1                  ;;; i = i + 1
  i = i + %2
  goto for3               ;;; endwhile
  label endfor3 :

  writeln

  return
endfunction
```

Example – Passing arrays per reference

```
;;;  This program reads a number x, fills an array of 10
;;;  elements with values x to x+9, passes it to a
;;;  function that reverses the array and computes the sum of
;;;  the elements,  and finally prints the array elements

function reverse_and_sum

  params
    _result integer
    b integer array
  endparams

  vars
    i integer
    n integer
    x integer
    s integer
  endvars

  ;;; reverse array, swapping x[i] with x[n-i-1] for all i<n/2
  ;;; and add all elements in s
  n = 10                  ;;; n=10
  s = 0                   ;;; s=0
  i = 0                   ;;; i=0
  label for2 :            ;;; while (i<n/2)
  %1 = 2
  %2 = n / %1
  %3 = i < %2
  ifFalse %3 goto endfor2
  %5 = b                  ;;; b is a pointer to a
  x = %5[i]               ;;; x = b[i]
  s = s + x               ;;; s = s + b[i]
  %1 = 1                  ;;; b[i] = b[n-1-i]
  %1 = n - %1
  %1 = %1 - i
  %2 = %5[%1]
  %5[i] = %2
  %5[%1] = x              ;;; b[n-1-i] = x

  s = s + %2              ;;; s = s + b[n-1-i]
```

```
  %2 = 1                  ;;; i = i + 1
  i = i + %2
  goto for2               ;;; endwhile
  label endfor2 :

  _result = s
  return
endfunction

function main
  vars
    x integer             ;;; int x, i, n, sum
    i integer
    n integer
    sum integer
    a integer 10          ;;; int a[10]
  endvars

  readi x
  n = 10                  ;;; n=10

  ;;; fill array with numbers from x to x+9
  i = 0                   ;;; i=0
  label for1 :            ;;; while (i<n)
  %1 = i < n
  ifFalse %1 goto endfor1

  %2 = x + i              ;;; a[i] = x+i
  a[i] = %2

  %2 = 1                  ;;; i = i + 1
  i = i + %2
  goto for1               ;;; endwhile
  label endfor1 :

  ;;; call function
  pushparam
  %1 = &a                 ;;; pass array address (reference parameter)
  pushparam %1
  call reverse_and_sum
  popparam
  popparam sum            ;;; sum = reverse_and_sum(a)

  writei sum              ;;; print sum
  writeln

  ;;; print array elements
  i = 0                   ;;; i=0
  label for3 :            ;;; while (i<n)
  %1 = i < n
  ifFalse %1 goto endfor3

  %2 = a[i]               ;;; write a[i]
  writei %2
  %2 = ' '                ;;; write ' '
  writec %2

  %2 = 1                  ;;; i = i + 1
  i = i + %2
  goto for3               ;;; endwhile
  label endfor3 :

  writeln

  return
endfunction
```

Description of the tests

In order to incrementally build the compiler, a list of program tests has been proposed. This table summarizes what are the new elements of the Asl language that appear in each test with respect to the previous one.

Some indications on modifying compiler components are also given, but should be taken *just as a guide*.

Tests for parsing and type check

The following tests can help you to define the grammar of the whole ASL language and to complete the semantic analysis (*SymbolsVisitor* and *TypeCheckVisitor* modules). Some changes in the ASL grammar will produce errors executing *g++* on *CodeGenVisitor.cpp*. Adjust this module when necessary.

Test	New structures / new checks	Changes in Asl grammar	Changes in SymbolsVisitor	Changes in TypeCheckVisitor	Adjust in CodeGenVisitor
`jp_chkt_01`	Other basic types, new operators, values, parenthesis, … Use of *type coercion*	Update rule *type* and rule *expr*. New tokens, floats, …	Update variable declaration (new basic types). See module *TypesMgr*	Type check of new expressions. *Type coercion* from *int* to *float* (update both .h and .cpp)	
`jp_chkt_02`	Multiple variable declarations in a single line. (new token *true*)	Update variable declaration	Update variable declaration	(new value *true*)	Fix access to the text of the *first* variable to avoid compilation error
Additional statements					
`jp_chkt_03`	*while* and *return* statements. *else* in conditionals. (new operator *!=*)	Update rule *statement*		Type check of new statements (update both .h and .cpp) Fix access to *statement* in *if* children (there are two now)	Fix access to *statement* children in *if* (there are two now) to avoid compilation error
Function declaration with parameters					
`jp_chkt_04`	Functions may have *parameters* and return values	Update function declaration (use new rules). Update return statement	Update *visitFunction* to create the appropriate function type. See module *TypesMgr*. **Hint:** add new visit methods to process parameters		
Array access					
`jp_chkt_05`	Array access in expressions and in *l-value* expressions	Update rules *left_expr* and *expr*		Add new visitors for array access in *left_expr* and in *expr* (update both .h and .cpp)	Modify the visitor for rule *left_expr* to use the new visit method
Function call					

Test	New structures / new checks	Changes in Asl grammar	Changes in SymbolsVisitor	Changes in TypeCheckVisitor	Adjust in CodeGenVisitor
`jp_chkt_06`	Call to function as a form of expression. Functions without parameters	Update rule *expr* to accept function calls		Add new visitors for function call. Type check the expression appropriately (update both .h and .cpp)	
`jp_chkt_07`	Call to function with parameters, without errors in arguments. No *main* function detection (automatic)			Type check arguments (*actual* parameters)	
`jp_chkt_08`	Call to procedures with parameters, without errors in arguments. Inappropriate calls to procedures and functions, regardless of arguments	Update statements		Type check of function call and compute expression type. Type check arguments (*actual* parameters)	
`jp_chkt_09`	Incorrect use of a function. *l-value* expressions in *read*			Compute properly the *IsLValue* decoration of the identifiers (probably nothing)	
Array declaration					
`jp_chkt_10`	Array declarations	New structured *type*. **Hint:** use 2 rules, *type* and *basic_type*	Add new visit methods to process types. See module *TypesMgr*. Note: remember that functions only return *basic* types (update both .h and .cpp)	Probably nothing	
`jp_chkt_11`	Use of *read* statement			Probably nothing if type check of array access is correct	
Return statement					
`jp_chkt_12`	Extensive use of *return* statement in procedures and functions, with and without *type coercion*.		Note: remember that the first declaration of an identifier *in a given scope* prevails over the rest (on *that scope*)	Update type check of *return* statement. **Hint:** the return type of the current function can be obtained from SymTab in *visitFunction*, and saved/retrieved with

Test	New structures / new checks	Changes in Asl grammar	Changes in SymbolsVisitor	Changes in TypeCheckVisitor	Adjust in CodeGenVisitor
				the methods *setCurrentFunctionTy* and *getCurrentFunctionTy* (available in TypeCheckVisitor).	
`jp_chkt_13`	Check undeclared identifiers				
Errors in parameters					
`jp_chkt_14`	Check that parameters must be declared in the *local scope*, with re-declarations of symbols in the same scope. Also, function calls may have errors due to the type of the *arguments*		Nothing if *visitFunction* was properly updated in the test `jp_chck_04`. Note: remember that the first declaration of an identifier *in a given scope* prevails over the rest (on that *scope*)	In a function call, remember that arguments (*actual parameters*) must always be processed. Now, also type check *actual* vs. *formal* parameters See module *SemErrors*	
`jp_chkt_15`	Similar to `jp_chck_14` and and previous ones: function calls with parameters and arrays involved				
`jp_chkt_16`	Similar to `jp_chck_15` and the previous ones: function calls with parameters and arrays involved				
`jp_chkt_17`	Similar to `jp_chck_16` and the previous ones: function calls with parameters, assignment to to an array, …				
`jp_chkt_18`	Function and procedure calls may have a wrong number of arguments			Type check function and procedure calls to detect also these errors. See module *SemErrors*	
`jp_chkt_19`	Use of operator *modulo* (%)	(Update rule *expr*)		Type check the operator *modulo*	
Operations on arrays					

Test	New structures / new checks	Changes in Asl grammar	Changes in SymbolsVisitor	Changes in TypeCheckVisitor	Adjust in CodeGenVisitor
`jp_chkt_20`	Array assignments, arrays as parameters, *type coercion*, ... Similar to previous tests			Probably nothing	

Tests for code generation

The following tests can help you to complete the code generation (*CodeGenVisitor* module). Some changes may involve your ASL grammar and/or type check. Adjust *Asl.g4* and *TypeCheckVisitor* module if necessary.

Test	New structures / new code	Adjust in Asl grammar	Adjust in TypeCheckVisitor	Changes in CodeGenVisitor
`jp_genc_01`	Multiple variable declarations in a single line. Other basic types, new operators, values, parenthesis,... *Type coercion* in some expressions. Extend `write` statement with new types			Update variable declaration. Update code for `write` statement. Generate code for new expressions, with *coercion* `int` → `float` in *arithmetic* operators. (Update both .h and .cpp). See module *code*
`jp_genc_02`	Use of `while` statement. New operators `>`, `<` (without *type coercion*). Boolean values `true` and `false`	If necessary, add value `false`	If necessary, type check the new value	Generate code for `while` statement, and for new expressions
Function call with parameters				
`jp_genc_03`	Functions may have *parameters* and return values. Call to a function as a form of expression (only parameters of *basic types*). Use of `return expr` statement (without *type coercion*)			Update function declaration (`visitFunction`). Add new visitors for function call, and generate code for this expression appropriately (update both .h and .cpp). See module *code*
`jp_genc_04`	Call to a procedure with *parameters* of *basic types* (without *type coercion*). New operator `<=` with *type coercion*			Update procedure call to allow *parameters*. Relational operators with *coercion* `int` → `float`
Array declaration and access				
`jp_genc_05`	Use of *array* type in declarations of local variables. Array access in expressions, and in *l-value* expressions. `if-then-else` statement. Function calls with *type coercion* in parameters	If necessary, complete the `if` statement with `else` branch	If necessary, complete the type check of the `if` statement	Update *assignment* statement: now a value can be assigned to an array position. Update `if` statement. Update function call to allow *coercion* in actual *parameters*

Test	New structures / new code	Adjust in Asl grammar	Adjust in TypeCheckVisitor	Changes in CodeGenVisitor
`jp_genc_06`	*Type coercion* in assignments, and in actual parameters of procedure calls. Use of `read` statement into non `int` expressions. Call to functions discarding the result (like a procedure)			Update *assignment* to allow *coercion*. Update `read` statement. Update code generation in “procedure” calls
Arrays as parameters				
`jp_genc_07`	Use of parameters of type `array`: access to the value and to the address of an array position. Procedure calls with actual parameters of type array (passed by *reference*)			Update procedure call to allow actual *parameters* of type *array* (passed by *reference*). Only local arrays are passed.
`jp_genc_08`	Function calls with actual parameters of type array (passed by *reference*)			Update function call to allow actual *parameters* of type *array* (passed by *reference*). Only local arrays are passed
`jp_genc_09`	Definition and use of the function `factorial`			Probably nothing
`jp_genc_10`	Definition and use of the function `prod_escalar` (*dot product* of two arrays)			Probably nothing, but check the pass of arrays as parameters
`jp_genc_11`	Extend `write` statement with *char* expression. Operator *modulo*. Values of type *char*			Update `write` statement. Add *modulo* operator in *arithmetic* expressions. Generate code for new values
Array assignment				
`jp_genc_12`	Array assignment (*a = b*), where *a,b* may be local variables and/or parameters			Update assignment statement
`jp_genc_13`	Additional checks of arrays passed as parameters, where the array is not the first param.			Probably nothing
`jp_genc_14`	Unary operator +			Generate code for this expression